

Report conversation

You are a Python expert.

How can I create a deep copy of a variable?

Certainly! In Python, if you want to create a deep copy of a variable, especially for complex objects like lists or dictionaries that contain nested objects, you should use the `copy` module which provides the `deepcopy` function.

Here's how you can use it:

python

 Copy code

```
import copy

original_list = [[1, 2, 3], [4, 5, 6]]
deep_copied_list = copy.deepcopy(original_list)

# Now, if you modify the nested list inside the original list...
original_list[0][0] = 99

print(original_list)          # Outputs: [[99, 2, 3], [4, 5, 6]]
print(deep_copied_list)      # Outputs: [[1, 2, 3], [4, 5, 6]] (remains uncl
```

The deep copy creates a new object that's a copy of the original, but recursively creates new objects for any mutable objects found inside the original. This ensures that changes to the original object or any of its nested objects won't affect the deep copied object.

On the other hand, if you used a shallow copy (e.g., using `copy.copy()` or for lists, simply doing `copied_list = original_list[:]`), changes to nested objects inside the original would reflect in the copied object as well.